

Predictive Home Automation Service

1. Purpose and Overview

The Predictive Home Automation Service learns how your household moves through your home each day and uses that knowledge to anticipate what will happen next. Rather than relying on fixed schedules or manual rules, it continuously observes occupancy patterns and builds an evolving picture of your routines — then puts that picture to work.

This service’s primary use case is assisted living and elder care. The same occupancy model that powers convenience automations (lighting, climate) is also what makes it possible to reliably tell the difference between “this room is just quiet right now” and “something may be wrong” — a distinction a fixed motion-timeout alarm cannot make, but a model that has learned your household’s actual rhythms can. §3 explains how that underlying learning works; §6 describes the assisted-living feature itself in full. The rest of this document applies to both use cases equally, since the same underlying model drives both.

In practical terms, this means the service can answer questions like:

- “Is an elderly or vulnerable family member moving through their home as expected today, or has it been unusually quiet?”
- “Is anyone likely to be in the living room in the next 15 minutes?”
- “Will the bedroom be occupied an hour from now?”

These answers are generated automatically and fed to the rest of your home automation platform, which can use them to control lighting, climate, security, and safety monitoring with minimal input from you.

2. How the Service Works

The service operates as a continuous background process. Here is what happens at each stage:

- 1. Sensor input.** Motion and presence sensors throughout your home send status updates whenever they detect a change — someone entering a room, leaving, or the room remaining empty. Each update is timestamped and stored locally.
- 2. Building a timeline.** The service converts those individual sensor readings into a minute-by-minute record of occupancy for each room. If a sensor reports presence at 9:14 AM and absence at 9:47 AM, the service fills in that 33-minute window accordingly. This creates a continuous, readable history of how each space has been used.
- 3. Building predictions from history.** The most recent 48 hours of each room’s timeline is the direct input to the prediction model. The model was trained on weeks of similar timelines from your home, so its output implicitly reflects patterns that recurred in that history — for example, whether a room tends to be occupied at a particular time of day. Each room’s *occupancy prediction* is generated independently — the model does not use one room’s timeline to predict another room’s occupancy. (The separate safety-monitoring feature described in §6 does look across rooms, for a different purpose.)
- 4. Generating predictions.** Using what it has learned, the service produces probability estimates for the *current* moment as well as two future time horizons:
 - **Right now** — is the room occupied at this moment? This is what drives immediate, responsive actions like turning on a light as someone enters a room.
 - **15 minutes** — near-term occupancy (useful for responsive lighting and climate control)
 - **1 hour** — medium-term occupancy (useful for pre-heating, pre-cooling, or security checks)

Each prediction is a number between 0 and 1 representing how likely it is that a room is or will be occupied. Your automation rules use these numbers to decide when to act.

5. Triggering automations. The platform evaluates each prediction against configured thresholds. When the probability of occupancy crosses a threshold, an action is triggered — for example, raising the thermostat, turning on lights, or disarming a sensor zone.

3. How the Service Learns

The service uses a machine learning model — a pattern-recognition engine — that is trained on your home’s own sensor history. It does not come pre-programmed with knowledge of your routines; it derives that knowledge from observation.

The learning process has three stages:

Stage	When it applies	What it means for you
Getting started	First day	Predictions are based on general templates

Getting started	First day	Predictions are based on general templates, not your home. Use for testing only; do not rely on them for critical automations.
Learning	Day 1 to Day 7	The model begins incorporating your real data. Predictions improve noticeably but may still vary. Manual overrides are recommended for important actions.
Mature	After Day 7	The model has enough data to reflect your actual patterns. Full automation is appropriate.

A worked example. Suppose your home office is reliably occupied on weekday mornings from roughly 8:30 AM to noon, and empty on weekends.

- **Getting started (Day 1):** Asked “will the office be occupied at 9 AM?”, the service has no home-specific data yet and falls back to a generic template — it might say something like 60% likely, regardless of whether it’s a Tuesday or a Saturday, because it has not yet seen your home’s own pattern at all.
- **Learning (Day 1-7):** By the middle of this stage, the service has trained on several real mornings. A weekday 9 AM query now correctly trends high (e.g. 75–85%). A *weekend* 9 AM query may still be inaccurate at first, simply because the model hasn’t yet seen enough Saturdays specifically to be confident about them — this is exactly why manual overrides are still recommended here.
- **Mature (Day 7+):** The service has now seen multiple full weeks. A weekday 9 AM query reliably returns a high probability; a weekend 9 AM query reliably returns a low one. The distinction between weekday and weekend mornings — not just “morning” in general — is what “learning your patterns” concretely means.

If your actual routine later changes — say, you start working from the office on Saturdays too — the nightly retraining described below means the model adapts automatically, without you needing to reconfigure anything. Because the training window looks back 14 days (roughly two occurrences of any given day of the week), a change tied to a specific day may take a few weeks of consistent repetition before it’s fully reflected, rather than showing up after a single occurrence.

The service evaluates which stage it is in automatically — you do not need to configure or trigger these transitions.

Adapting over time. Once mature, the model continues to update itself on a nightly basis (between 11 PM and 2 AM by default, when the household is typically inactive). This means it naturally adjusts to changes in your routine — a new work-from-home schedule, seasonal shifts in activity, or a change in household membership — without any manual reconfiguration required.

Two components, two learning styles. Underneath the predictions described in §2 sit two different pieces of machine learning, and they learn in different ways — worth understanding so you know what to expect. A shared foundation model is retrained from its original starting point every single night, using the most recent 14 days of your data each time. Its role is to build a general understanding of your home’s occupancy patterns — it is not what makes the final call your automations act on. That final call — the **right now, 15-minute**, and **1-hour** predictions described in §2 — is produced by three small, specialized components built on top of that foundation, one per case. Unlike the foundation model, these three do not start over each night: each refines what it already learned the night before, a little further. In practice this means all three of these predictions tend to become steadily more precise the longer the service runs, well past the “mature” milestone in the table above — including the “right now” prediction that drives your most immediate automations. This split exists deliberately — restarting the larger foundation model each night keeps it from gradually drifting over months of continuous updates, while the much smaller components built on top of it can safely keep building on their own progress.

4. Data Window and History

The service uses two different data windows for two different purposes. Understanding the distinction helps set accurate expectations about what the model knows and how quickly it adapts.

Prediction window — 48 hours. When generating any prediction, the service queries the most recent 48 hours of sensor readings for the room being evaluated. This 48-hour timeline is the direct input to the prediction model, and it is the same window length regardless of the forecast horizon — whether predicting occupancy right now, in 15 minutes, or an hour ahead.

Training window — 14 days. Each nightly training run draws on the most recent 14 days of stored sensor events. From those 14 days, the pipeline generates a large set of training examples by sliding a 48-hour input window across the full timeline in 15-minute increments. Each individual training example therefore still contains only 48 hours of input context — the 14-day window simply provides enough history to generate a varied and representative set of those examples, covering multiple weekday and weekend cycles.

These two windows serve different purposes. The **48-hour prediction window** gives the model current context at inference time — what has actually been happening in your home over the past two days. The **14-day training window** ensures the model has seen enough of your home’s weekly rhythms to understand what is typical, without being anchored to behaviour from further back that may no longer reflect your current routine.

The database retains sensor event data for 30 days and then discards it. No raw event data is kept beyond that retention period. (This 30-day retention period is separate from, and longer than, the 14-day window used for training — it exists to give the training window room to look back safely without ever bumping against the edge of what’s actually stored.)

5. Data Reuse and Continuity

Data is accumulated, not discarded between training cycles. Within the 14-day training window, the same sensor events can be used across multiple nightly training runs. Before each run, the service makes the full two weeks of available events accessible again — meaning each training run benefits from the complete recent history, not just the events that arrived since the previous run.

When retraining happens. Retraining is triggered on a nightly schedule, during the quiet window between 11 PM and 2 AM. It can also be triggered by significant events, such as when the system advances from the “learning” phase to the “mature” phase. The process is fully automatic and requires no action from you.

Vacation Mode — suppressing absence alerts (manual). If you know you will be away, you can enable Vacation Mode using the button in the Monitor interface. This is specifically an **absence-alert suppression switch**. When enabled, it tells the anomaly detector that a prolonged period of silence in your home is expected, preventing false safety notifications during a planned absence.

Vacation Mode does **not** pause model training, stop data collection, or affect predictions in any way. The service continues to record sensor events and run its scheduled nightly training as normal while you are away. Vacation Mode should be enabled before you leave and disabled when you return.

Protecting training quality on unusual days (automatic). Separately from Vacation Mode, the training pipeline includes an automatic mechanism that detects when the current data would teach the model the wrong patterns — for example, during an unannounced holiday, a period of illness, or any stretch of days when activity is unusually low compared to your normal routine for that day of the week.

Before each training run, the system compares the day’s aggregate activity level against a statistical baseline built from the same day of the week over the preceding weeks. If the activity level has been significantly below normal for three or more consecutive days, the system concludes that the period is anomalous and either skips the training run entirely (once the model is mature) or blends in synthetic reference data to compensate (during the learning phase). This protects the model from learning that an empty house is the norm.

This mechanism is entirely separate from Vacation Mode. It runs inside the training pipeline and reads no information from the Vacation Mode setting. Enabling Vacation Mode does not trigger it, and it has no effect on anomaly alerts. The two systems solve different problems:

- **Vacation Mode** is a manual control you activate to silence safety alerts when you know you are leaving.
- **Automatic holiday detection** is a background safeguard that protects the model’s long-term accuracy when unusual absence is detected, with no manual input required.

Model consistency. The prediction model and its associated forecast heads (one per time horizon) are always kept in sync. If a training run produces a model that fails quality checks, the previous model remains in service. If a successful update is later rolled back for any reason, the forecast heads roll back with it.

6. Privacy and Data Scope

What is collected. The service collects only motion and presence events from the sensors you have configured. Each event contains a room identifier, a timestamp, and a presence state (occupied or unoccupied). No audio, video, or personally identifiable information is collected.

Where data is stored. All sensor data, learned models, and configuration are stored locally on your Aragon Maestro device. Nothing is sent to a cloud server or external service. Your occupancy history never leaves your home network.

What is not collected.

- The service does not record who specifically is present in a room — only that the room is occupied.
- It does not monitor the content of conversations, activities, or behaviour beyond binary presence detection.
- It does not connect to third-party data sources.

Assisted living monitoring (if enabled) — the service’s primary use case (see §1). For homes where an assisted living safety feature is active, the service continuously monitors for unusually long periods of inactivity across all configured rooms. What counts as “unusual” is not a fixed timer — it is informed by the same learned occupancy model described throughout this document, which is why the system can distinguish a genuinely concerning silence from an ordinary quiet afternoon.

If an extended silence is detected — meaning no sensor activity has been recorded for longer than expected — a notification alert is issued. Before issuing an alert, the service checks whether any *other* monitored room has shown genuine recent activity; if so, the alert is held back, since a person being active elsewhere in the home is itself evidence they are safe, even if one particular room has been quiet. This reduces false alarms — for example, a room whose only sensor is a light switch that simply wasn’t needed during daylight hours won’t trigger a false safety alert on its own.

This feature can be enabled, disabled, or paused at any time through the platform settings, and — consistent with everything else in this document — operates entirely on locally stored data, with every alert generated on-device.

Document version 1.1 — Corrections applied: cross-room pattern claim removed; prediction vs. training data windows clarified and separated; Vacation Mode scope corrected to alert-suppression only; automatic holiday detection separated from manual Vacation Mode with explicit comparison.

Document version 1.2 — Corrections applied: raw data retention corrected from 7 to 30 days; training-reuse window corrected from 7 to 14 days (and clearly distinguished from the now-different retention period, since these were previously the same number and are no longer); removed the 2-hour prediction horizon, which is no longer produced by the system (15-minute and 1-hour horizons remain).

Document version 1.3 — Corrections applied: §2’s “no relationship between rooms” claim narrowed to occupancy predictions specifically, since the assisted-living safety feature now does check across rooms (added below); §6 updated to describe this cross-room check and why it exists (reduces false safety alarms). Added a concrete worked example to §3 illustrating what changes at each learning stage, and softened an earlier draft’s specific “1–2 week” adaptation-speed claim to avoid asserting a precision not actually verified.

Document version 1.4 — Corrections applied: elevated assisted living / elder care to its explicitly stated primary use case (§1 opening, a leading example question, and an expanded §6 treatment with cross-reference back to §1) rather than a brief mention buried in the privacy section. Added a fact-checked explanation to §3 of a real architectural distinction: a shared foundation model is retrained from scratch every night (14-day window each time), while three specialized components built on top of it — powering the “right now,” 15-minute, and 1-hour predictions — accumulate learning incrementally instead of restarting. (An earlier draft of this paragraph incorrectly implied “right now” used the from-scratch foundation model directly; corrected after verifying against source that it’s produced by one of the three accumulating components, same as the 15-minute and 1-hour predictions.) §2 correspondingly updated to list “right now” explicitly alongside the two forecast horizons, rather than only describing future-looking predictions. Replaced “home hub” with “Aragon Maestro device” for product-name accuracy.

Document version 1.5 — Addition: one sentence added to §3 giving a brief, high-level reason for the cold-start/accumulate split, so the distinction doesn’t read as arbitrary. Kept deliberately short and free of implementation detail, consistent with this document’s scope — see the developer documentation for the fuller engineering rationale.